

1. Introduction

Dans le cadre du projet de classification d'images *Food-11*, l'objectif est de concevoir, entraîner et comparer différents réseaux de neurones convolutionnels (CNN) qui sont capables de reconnaître automatiquement des catégories d'aliments à partir d'images.

L'évaluation des performances est réalisée sur un serveur externe à l'aide du *F1-score*, ce qui garantit une comparaison équitable entre les modèles soumis.

La démarche suivie s'articule autour de deux grandes étapes :

1. la mise en place et l'entraînement d'un réseau convolutionnel de type *LeNet-5*, conçu et entraîné directement sur le dataset *Food-11*,
2. puis la comparaison de ses performances avec des modèles pré-entraînés issus du transfert d'apprentissage (tels que *ResNet-18*).

L'objectif final est d'évaluer les limites d'un apprentissage direct "*from scratch*" sur un jeu de données de taille moyenne, et de démontrer l'intérêt du transfert de connaissances à partir de réseaux profonds déjà entraînés sur des bases massives comme *ImageNet*.

Le jeu de données *Food-11* contient au total 16 393 images réparties en 11 classes correspondant à différentes catégories d'aliments : *bread*, *dairy*, *dessert*, *egg*, *fried food*, *meat*, *noodles-pasta*, *rice*, *seafood*, *soup* et *vegetable-fruit*.

Parmi celles-ci, 13 296 images sont utilisées pour l'entraînement et 3 097 pour le test final.

Afin d'évaluer les performances du modèle pendant l'apprentissage, nous avons choisi de séparer le jeu d'entraînement en deux sous-ensembles : 90 % des images sont dédiées à l'entraînement effectif, 10 % sont réservées à la validation.

Cette organisation permet de suivre la capacité du modèle à généraliser tout en évitant le surapprentissage.

Il est important de noter que nous sommes face à un problème de déséquilibre de classe, se problème sera traité dans la partie 3 :

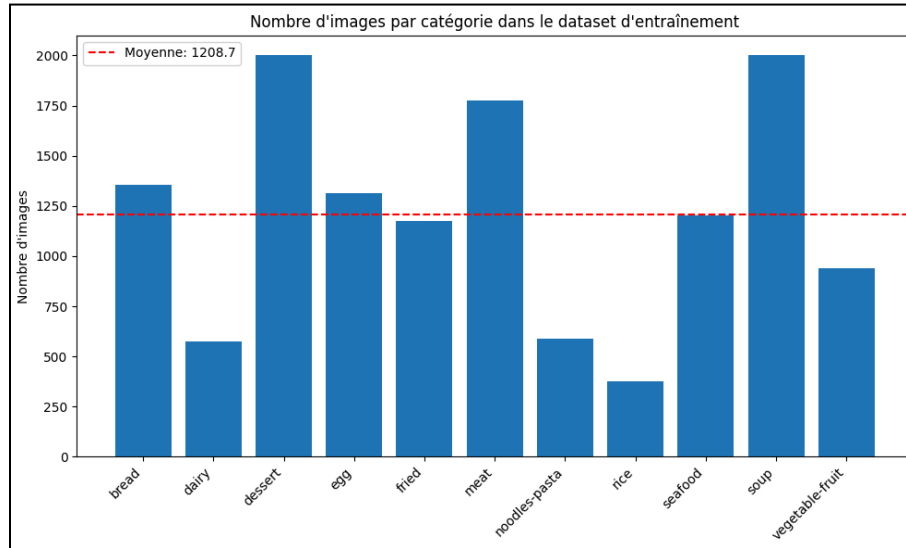


Figure 1 : Nombre d'images dans le dataset d'entraînement en fonction des classes

2. Première approche – Réseau CNN *LeNet-5*

Afin d'établir une première référence de performance, nous avons commencé par entraîner un réseau convolutionnel classique : *LeNet-5*.

À la base, ce modèle a été conçu pour reconnaître des chiffres manuscrits sur la base *MNIST* (images en 28×28), et même s'il est très simple, il reste efficace. Il est composé de trois couches convolutionnelles (6 filtres 5×5 , puis 16, puis 120), chacune suivie d'une fonction d'activation *ReLU* et d'un *average pooling* pour réduire la taille des cartes d'activation.

L'objectif de cette première expérience était de mesurer la capacité de ce réseau, sans aucune optimisation particulière, à extraire des caractéristiques discriminantes sur le dataset *Food-11*.

Les images ont volontairement été gardées en 512×512 (taille majoritairement présente dans le dataset mais pas toutes) afin de conserver un maximum de détails visuels.

Aucun prétraitement (pas de normalisation avancée, ni de régularisation, ni d'augmentation de données) n'a été appliqué à ce stade : le réseau apprend directement à partir des images du dataset.

Résultats

L'entraînement du réseau *LeNet-5* a été réalisé sur des images redimensionnées en 512×512 , avec un taux d'apprentissage initial fixé à 1×10^{-3} , un *mini-batch size* de 32 et un total de 5 époques. L'optimiseur *Adam* a été privilégié car il converge beaucoup plus rapidement que le SGD (*Stochastic Gradient Descent*) classique sur des jeux de données de taille intermédiaire comme le *Food-11* (environ 11 000 images après découpe *train/validation*). Grâce à son ajustement adaptatif du taux d'apprentissage, *Adam* stabilise les gradients même lorsque les classes présentent des variabilités visuelles fortes, ce qui est en fait un choix particulièrement efficace dans notre contexte.

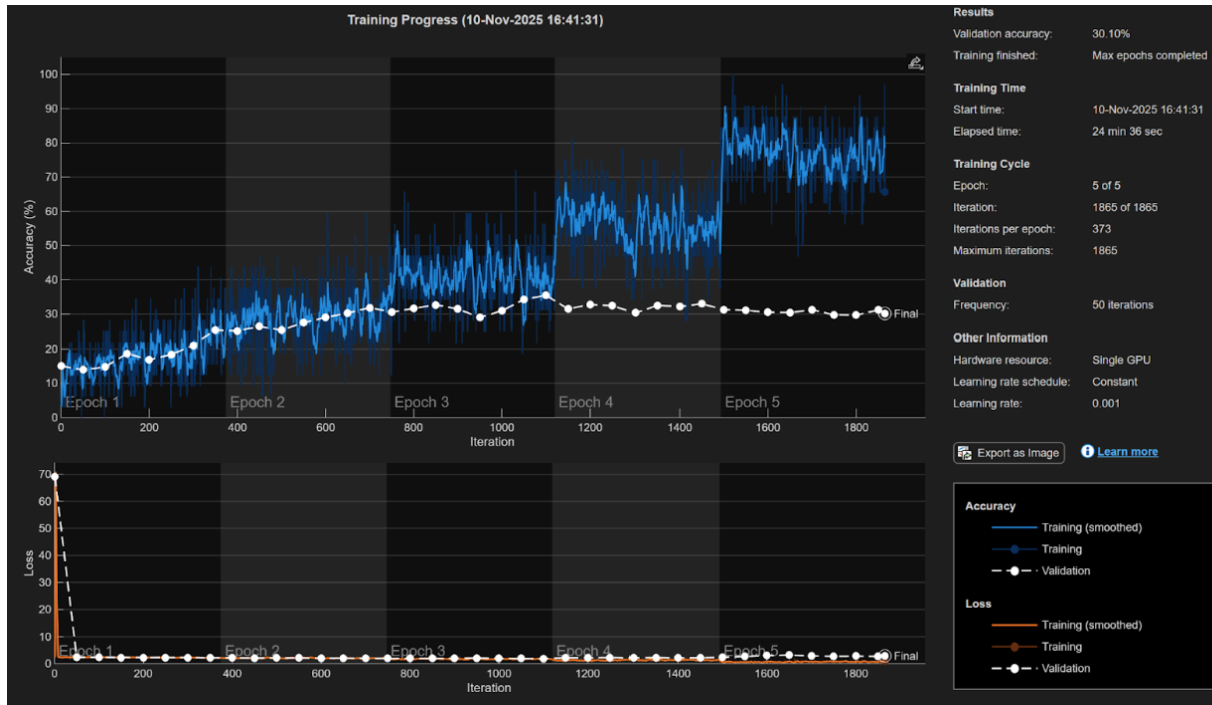


Figure 2 : Processus d'entraînement de LeNet-5 "brut" (512×512)

À l'issue de cette phase, le modèle atteint une *accuracy* de validation d'environ 30 %, tandis que la précision sur l'ensemble d'entraînement dépasse 70 %. Cette différence marquée entre les deux courbes traduit un phénomène clair de sur-apprentissage : le réseau parvient à mémoriser les exemples du jeu d'entraînement, mais échoue à généraliser efficacement sur des données inédites. Cette explosion se remarque fortement à partir de la 4^e époque.

L'analyse de la matrice de confusion confirme cette tendance:

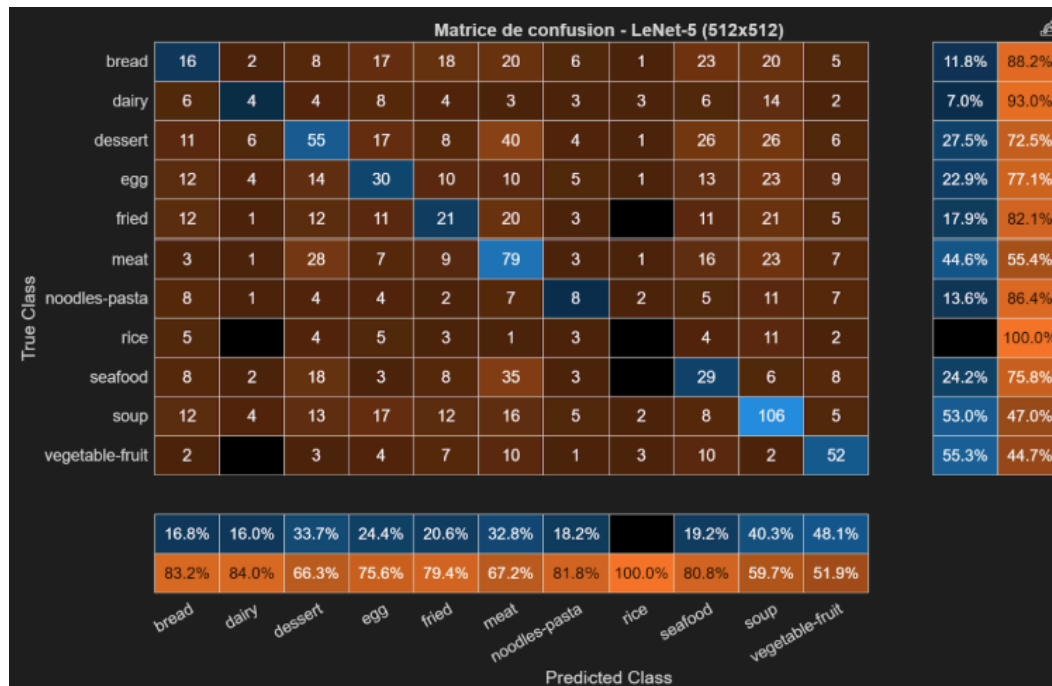


Figure 3 : Matrice de confusion de LeNet-5 "brut" (512×512)

Certaines classes, comme *vegetable*, *meat* ou *soup*, sont relativement bien reconnues, tandis que d'autres, telles que *rice*, *dairy*, etc, présentent de fortes confusions. Pour information le modèle après 5 itérations n'arrive pas à reconnaître le riz. Ces erreurs peuvent s'expliquer en partie par la similarité visuelle entre certaines catégories d'aliments, notamment dans les plats composés, où plusieurs ingrédients partagent des textures ou des couleurs proches (par exemple, viande et légumes dans un même plat).

Sur le plan technique, les performances limitées du modèle s'expliquent principalement par deux facteurs étroitement liés à l'architecture et à la taille d'entrée choisie.

Tout d'abord, la résolution élevée des images (512×512) impose un volume de calcul considérable pour un réseau aussi peu profond. Après seulement deux couches de *pooling*, la taille des cartes de caractéristiques atteint encore 128×128×120 activations, soit environ 1,96 million de valeurs reliées à la première couche *fully-connected* de 84 neurones. Le nombre total de paramètres s'élève ainsi à près de 165 millions, ce qui est largement disproportionné par rapport à la taille du dataset (environ 13 000 images). Cette complexité excessive conduit à un entraînement long, une forte consommation de mémoire et un risque accru de sur-apprentissage.

Par ailleurs, la simplicité de l'architecture du *LeNet-5* limite sa capacité à extraire des caractéristiques pertinentes sur des images aussi variées et riches visuellement que celles du *Food-11*. Le modèle, conçu à l'origine pour des images de faible résolution (32×32 sur la base MNIST), peine à capturer la diversité des textures, des formes et des compositions présentes dans ce dataset.

Amélioration du modèle

Suite aux résultats limités obtenus avec le modèle *LeNet-5* “brut”, nous avons décidé de conserver la même architecture afin d’en évaluer le potentiel, mais en introduisant plusieurs optimisations simples et progressives. L’objectif de cette seconde phase n’était pas de concevoir un nouveau réseau, mais plutôt de comprendre comment améliorer la généralisation d’un modèle de base avant de passer à des architectures pré-entraînées plus complexes.

La première modification a consisté à réduire la taille des images d’entrée de 512×512 à 128×128 pixels. Ce changement vise à alléger considérablement la charge de calcul tout en limitant la perte d’informations visuelles importantes. Avec cette nouvelle résolution, le volume des données en sortie du dernier bloc convolutionnel est ramené à $32 \times 32 \times 120$, soit environ 122 000 activations. Le nombre total de paramètres chute ainsi à quelques millions, rendant le réseau beaucoup plus adapté à un dataset de taille moyenne comme *Food-11*.

Les images ont ensuite été normalisées sur l’intervalle $[0, 1]$ afin d’uniformiser les intensités de pixels et de stabiliser la phase d’optimisation.

Pour renforcer la robustesse du modèle, une augmentation des données a été introduite. Contrairement à la première approche où les images étaient utilisées telles quelles, chaque image d’entraînement subit désormais des transformations aléatoires combinant des augmentations géométriques (rotations de $\pm 15^\circ$, translations horizontales et verticales) et photométriques (légères variations de luminosité, de contraste et de saturation).

Ces transformations sont appliquées de manière dynamique à chaque itération, ce qui permet au modèle de “voir” une version différente de chaque image à chaque époque, sans augmenter physiquement la taille du dataset. Cela permet d’obliger le réseau à se concentrer sur des caractéristiques invariantes (comme les formes, textures, couleurs moyennes, etc) plutôt que sur des détails spécifiques à chaque image, ce qui permet de limiter le sur-apprentissage.

Résultats et analyse comparative

L’entraînement du modèle *LeNet-5* optimisé (images 128×128 avec augmentation de données) a été réalisé sur 5 époques, un taux d’apprentissage de 1×10^{-3} et un *mini-batch size* de 64. Cette valeur a été retenue car elle constitue l’équilibre le plus efficace entre stabilité du gradient et contraintes matérielles : en-dessous, l’apprentissage devenait plus bruyé et instable, et au-delà de 64, la mémoire disponible ne permettait plus de traiter les images en parallèle sans saturation (limite matériel). Le suivi de l’apprentissage montre une évolution plus stable de la précision, avec une *accuracy* de validation atteignant 37,5 %, soit un gain d’environ 7 points par rapport au modèle initial (30 % avec les images 512×512). Le temps d’entraînement est resté comparable (≈ 25 minutes), mais la convergence du modèle s’est révélée plus progressive et mieux régulée.

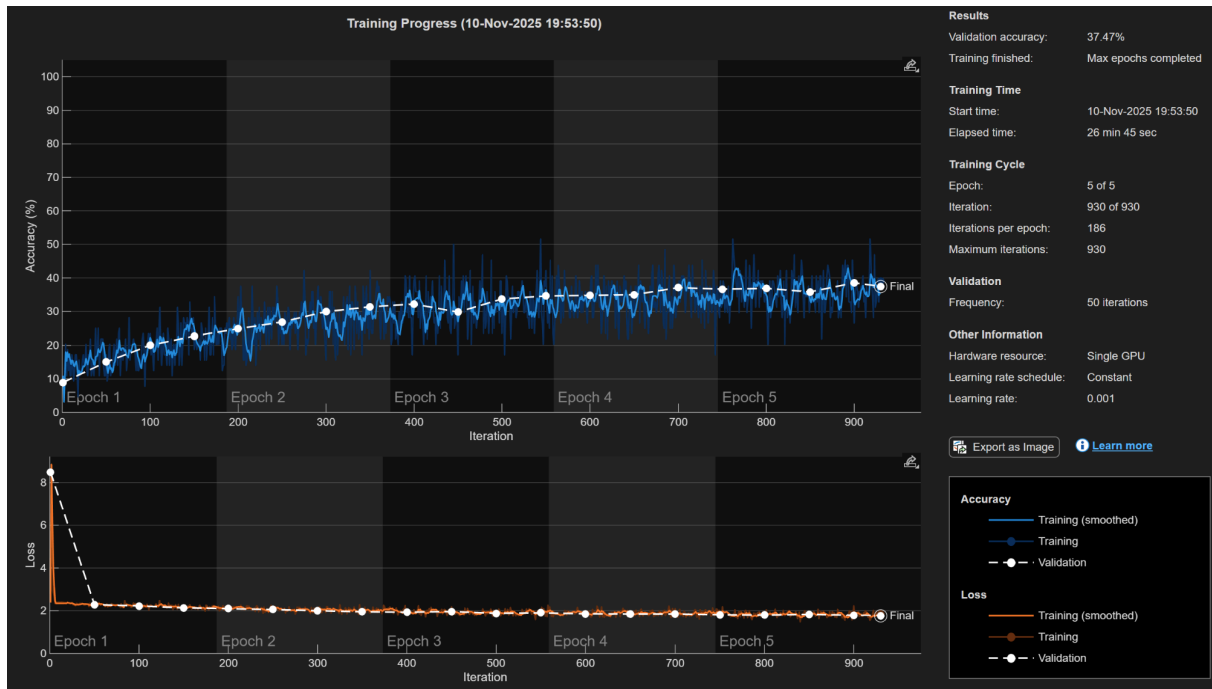


Figure 4 : Processus d'entraînement de LeNet-5 (128×128 + augmentation)

On observe sur la courbe d'apprentissage une progression plus linéaire de l'*accuracy*, sans la divergence marquée entre l'entraînement et la validation constatée dans la première version. La perte (*loss*) diminue rapidement au début, puis se stabilise, traduisant un apprentissage plus homogène et moins sujet au sur-ajustement.

L'analyse de la matrice de confusion confirme cette amélioration :

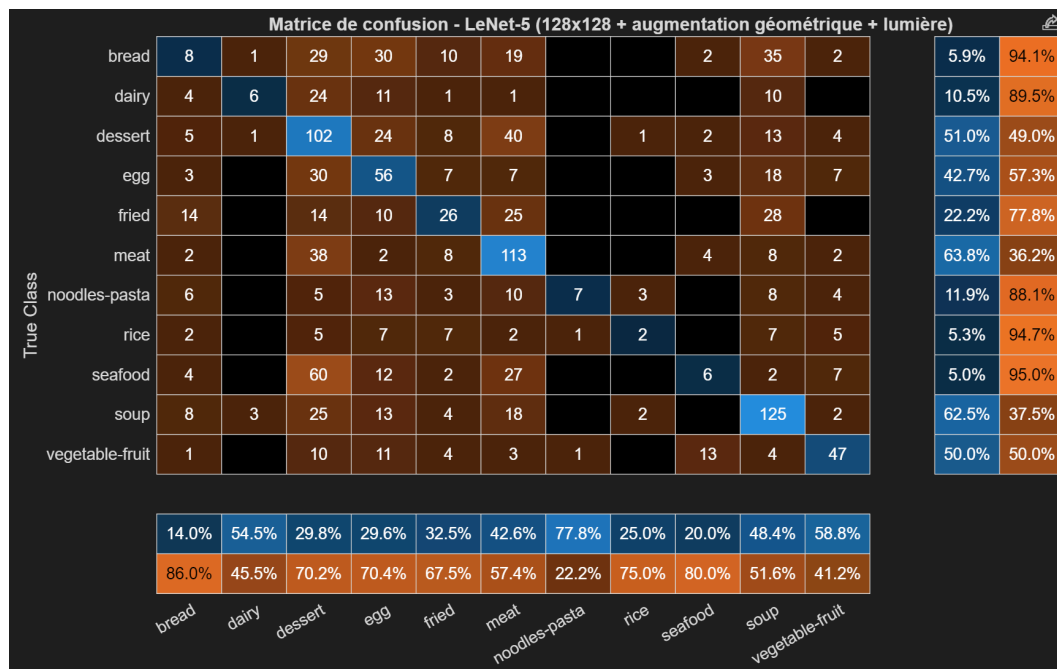


Figure 5 : Matrice de confusion de LeNet-5 (128×128 + augmentation)

On remarque que certaines classes, comme *dessert*, *meat* ou *soup*, atteignent des taux de reconnaissance nettement supérieurs à la moyenne, tandis que d'autres, telles que *rice*, *seafood* ou *noodles-pasta*, restent mal classées.

Cette hétérogénéité souligne que le réseau parvient à extraire des motifs caractéristiques pour certaines catégories bien représentées ou visuellement distinctes, mais peine à généraliser lorsque les classes présentent des textures ou des couleurs proches.

Globalement, le modèle conserve des performances modestes, mais la tendance générale est positive : la stabilité de l'entraînement s'améliore, la variance entre les classes se réduit légèrement, et la précision moyenne progresse de quelques points.

Cette expérience montre qu'en optimisant simplement le prétraitement et la préparation des données, il est possible d'obtenir un modèle plus robuste sans modification de l'architecture.

3. Réseaux pré-entraînés – Stratégie de transfert

Après avoir étudié les limites du *LeNet-5*, même optimisé en 128×128 avec augmentation de données, nous avons adopté une approche de transfert d'apprentissage (*transfer learning*). L'idée est de partir d'un réseau convolutionnel profond déjà entraîné sur une base d'images très large (*ImageNet*), puis d'adapter uniquement la fin du modèle aux 11 classes du *Food-11*. Cette stratégie permet de bénéficier de filtres déjà capables de détecter des motifs visuels génériques (bords, textures, formes) tout en réduisant le volume de données nécessaires pour atteindre de bonnes performances.

Le premier modèle testé dans ce cadre est *ResNet-18*, un réseau résiduel de 18 couches, bien adapté aux jeux de données de taille moyenne. Les images du *Food-11* sont redimensionnées en $224 \times 224 \times 3$, taille d'entrée native du réseau. La dernière partie du modèle (couche *fully-connected*, *softmax* et couche de classification) est remplacée par un nouveau bloc produisant 11 sorties, correspondant aux classes du challenge.

L'entraînement du modèle *ResNet-18* a été effectué sur les images redimensionnées en $224 \times 224 \times 3$, avec un taux d'apprentissage initial de 1×10^{-4} , un *batch size* de 64, et l'optimiseur *Adam*. Le modèle a été entraîné sur 10 époques, dans le cadre d'un transfert d'apprentissage partiel. Concrètement, le réseau *ResNet-18* pré-entraîné sur *ImageNet* a été conservé dans sa quasi-totalité : les couches convolutionnelles internes, responsables de l'extraction de caractéristiques générales (bords, textures, formes), ont été gelées afin de préserver les filtres déjà appris sur des millions d'images. Seule la tête de classification composée de la couche *fully-connected*, du *softmax* et de la couche de classification finale a été réentraînée sur les 11 classes du dataset *Food-11*. Cette approche permet d'adapter le réseau à la nouvelle tâche tout en limitant le nombre de paramètres à ajuster, ce qui garantit ainsi une convergence rapide et stable même avec un volume de données modéré.

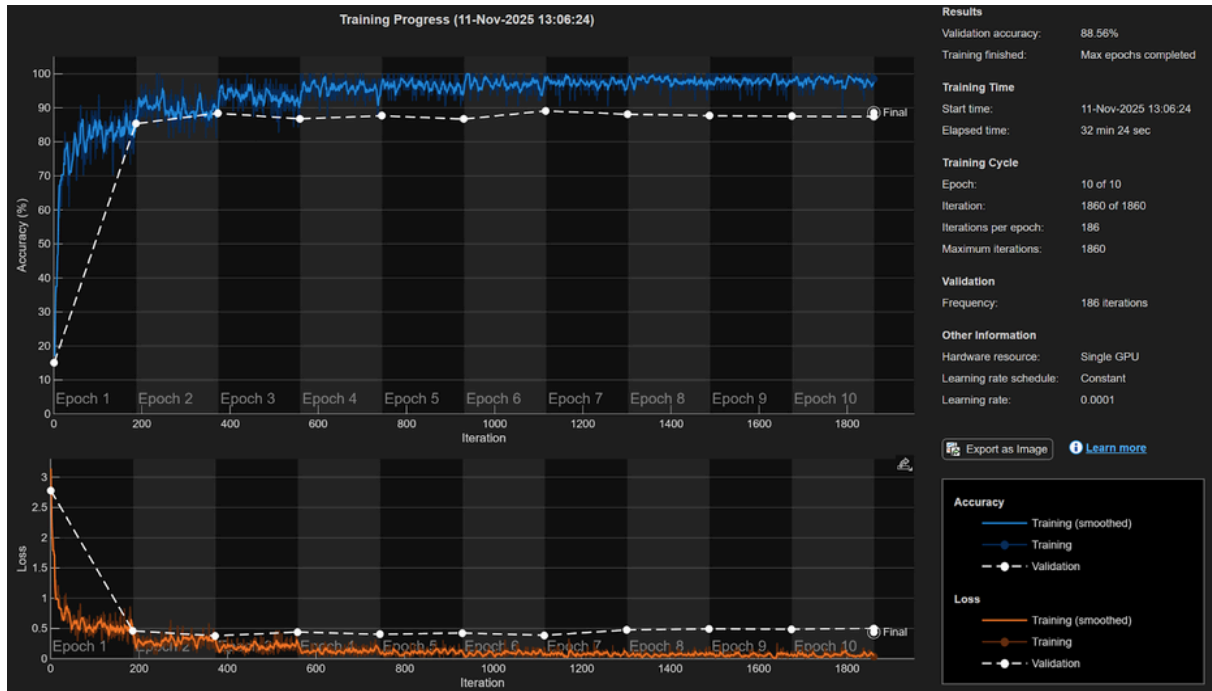


Figure 6 : Processus d'entraînement de *ResNet-18* (Transfer Learning sur *Food-11*)

La courbe d'apprentissage montre une progression rapide de l'*accuracy*, atteignant près de 88.56% sur l'ensemble de validation dès la 3^e époque, puis se stabilisant autour de cette valeur. L'*accuracy* d'entraînement avoisine 97 %, ce qui traduit une excellente capacité d'adaptation du modèle tout en conservant une bonne généralisation. L'écart modéré entre les deux courbes ($\approx 10\%$) indique que le réseau présente un sur-apprentissage.

Matrice de confusion - ResNet-18 (transfert learning Food-11)											
True Class	bread	dairy	dessert	egg	fried	meat	noodles-pasta	rice	seafood	soup	vegetable-fruit
	113	46	177	111	102	152	59	34	110	184	89
	83.1%	16.9%	88.5%	84.7%	87.2%	85.9%	100.0%	89.5%	91.7%	92.0%	94.7%
	1	5	4	2	4	6	1	2	3	1	2
	80.7%	19.3%	11.5%	15.3%	12.8%	14.1%	10.5%	10.5%	8.3%	8.0%	5.3%
	7	4	4	3	2	2	2	2	2	3	2
	88.5%	88.5%	88.5%	88.5%	88.5%	88.5%	88.5%	88.5%	88.5%	88.5%	88.5%
	3	1	4	2	1	1	1	1	1	1	1
	84.7%	84.7%	84.7%	84.7%	84.7%	84.7%	84.7%	84.7%	84.7%	84.7%	84.7%
	4	1	4	4	4	4	4	4	4	4	4
	87.2%	87.2%	87.2%	87.2%	87.2%	87.2%	87.2%	87.2%	87.2%	87.2%	87.2%
	3	4	4	6	10	1	1	1	1	1	1
	85.9%	85.9%	85.9%	85.9%	85.9%	85.9%	85.9%	85.9%	85.9%	85.9%	85.9%
	3	1	1	1	1	1	1	1	1	1	1
	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%
	1	1	1	1	1	1	1	1	1	1	1
	89.5%	89.5%	89.5%	89.5%	89.5%	89.5%	89.5%	89.5%	89.5%	89.5%	89.5%
	1	1	1	1	1	1	1	1	1	1	1
	91.7%	91.7%	91.7%	91.7%	91.7%	91.7%	91.7%	91.7%	91.7%	91.7%	91.7%
	1	1	1	1	1	1	1	1	1	1	1
	92.0%	92.0%	92.0%	92.0%	92.0%	92.0%	92.0%	92.0%	92.0%	92.0%	92.0%
	1	1	1	1	1	1	1	1	1	1	1
	94.7%	94.7%	94.7%	94.7%	94.7%	94.7%	94.7%	94.7%	94.7%	94.7%	94.7%
Predicted Class											

Figure 7 : Matrice de confusion de *ResNet-18* (Transfer Learning sur *Food-11*)

Le modèle *ResNet-18* atteint des performances très satisfaisantes sur la majorité des classes, avec des taux de reconnaissance supérieurs à 85 % pour la plupart d'entre elles. En revanche, certaines catégories présentent encore des confusions notables, en particulier *dairy*. Cette classe regroupe des aliments très variés visuellement, tels que le lait, les yaourts, les fromages, les crèmes ou encore les sauces blanches, souvent photographiés sur des fonds clairs. Ce manque d'uniformité interne rend l'apprentissage difficile : le réseau peine à identifier une texture ou une forme dominante propre à cette catégorie. Un autre point intéressant concerne la classe *rice*, qui, bien que sous-représentée dans le dataset, obtient paradoxalement le meilleurs taux de reconnaissance (89.5 % sur l'ensemble de validation). Ce résultat peut s'expliquer par la forte homogénéité visuelle des images associées à cette catégorie : le riz présente des motifs répétitifs, une texture fine et une couleur claire, qui contrastent fortement avec celles des autres plats.

Modèle \ Métrique	Hyperparams clés	Temps d'entraînement	Accuracy	Precision	Recall	F1-score
ResNet-18	LR = 1×10^{-4} BS = 64 10 époques	≈ 32 min	0.9083	0.8060	0.9286	0.8629
ResNet-50		≈ 120 min	0.9303	0.8822	0.9444	0.9122
MobileNet-V2		≈ 60 min	0.9248	0.9032 (meilleur)	0.9268	0.9148
EfficientNet-B0		≈ 114 min	0.9399 (meilleur)	0.8967	0.9558 (meilleur)	0.9253 (meilleur)

Figure 8 : Tableau comparatif de différents modèles pré-entraîné sur le serveur de test

De manière générale, les erreurs de classification observées sont cohérentes d'un modèle à l'autre et concernent principalement les classes *dairy* et *bread*. La catégorie *dairy* souffre avant tout d'un déséquilibre dans le dataset, avec un nombre d'images plus limité et une forte variabilité interne. Les produits laitiers regroupent des aliments très différents visuellement tels que les yaourts, les fromages, les sauces ou le lait, ce qui rend difficile l'apprentissage d'une texture ou d'une forme commune. La classe *bread*, pourtant bien représentée, pose un autre type de difficulté : ses textures (croûte dorée, surfaces farineuses) et ses teintes proches de certaines pâtisseries ou aliments frits entraînent des erreurs d'association visuelle. Ces confusions ne traduisent donc pas un défaut du modèle, mais plutôt les limites intrinsèques du dataset, où les frontières entre certaines catégories restent floues. À titre expérimental, nous avons également testé une version du *ResNet-18* sans augmentation photométrique (pas de variation de luminosité, de contraste ou de saturation). Les résultats obtenus sont très proches, avec une *accuracy* de 87.43 % contre 88.56 % pour le modèle complet, mais pour un temps d'entraînement réduit à seulement 14 minutes, soit près de deux fois plus rapide. Ce test montre que la photométrie n'apporte qu'un gain marginal sur la précision globale, mais augmente sensiblement le coût en calcul.

Amélioration

Dans le but d'obtenir de meilleurs résultats, nous allons adopter une approche d'amélioration ciblée plutôt que globale. L'analyse des matrices de confusion a montré que certaines classes, comme *rice* ou *noodles-pasta*, sont déjà bien reconnues malgré leur faible représentation, tandis que d'autres, notamment *dairy* et *fried food*, restent difficiles à distinguer. Plutôt que de modifier la distribution du dataset (interdiction d'après le sujet), nous concentrerons nos ajustements sur ces classes problématiques. Concrètement, nous renforcerons leur influence pendant l'entraînement en pondérant la fonction de perte (*class weights*) de manière à ce que les erreurs sur ces catégories comptent davantage. En parallèle, une augmentation photométrique sélective (variations légères de luminosité, de contraste et de rotation) sera appliquée uniquement aux images de ces classes, afin d'accroître la diversité visuelle perçue par le réseau tout en diminuant le temps d'entraînement. Enfin, nous avons pu réaliser des essais complémentaires grâce à la gentillesse d'un camarade qui nous a donné temporairement accès à un ordinateur plus performant. Cela nous a permis d'augmenter la résolution d'entrée de 128×128 à 224×224 pour les modèles pré-entraînés. Rien que cette modification, sans changer aucun autre hyperparamètre, a entraîné une hausse d'environ 1 % de l'*accuracy* sur l'ensemble de validation.

Nous obtenons à la fin un score remarquable de :

Métrique	Accuracy	Precision	Recall	F1-Score
Valeur	0.9561	0.9263	0.9612	0.9435

Figure 9 : Performance finale sur le serveur en ligne avec *EfficientNet-B0*

Plusieurs pistes d'amélioration auraient également pu être explorées pour optimiser davantage les performances du modèle. Par exemple, l'utilisation d'une optimisation bayésienne des hyperparamètres (*Bayesian Optimization*) aurait permis d'ajuster automatiquement le taux d'apprentissage, la profondeur du *fine-tuning* ou la force de la régularisation afin de converger vers une configuration plus optimale. De même, l'introduction d'une régularisation *L2* ou d'un *scheduler* de taux d'apprentissage plus sophistiqué (par exemple un double taux d'apprentissage ou un *scheduler* de type *cosine annealing*) aurait probablement contribué à stabiliser davantage l'entraînement et à réduire le sur-ajustement observé sur certaines architectures. Toutefois, ces expérimentations nécessitent un temps de calcul important et une exploration systématique que notre planning ne permettait pas d'inclure dans ce projet. Elles constituent néanmoins des perspectives intéressantes pour un travail futur ou une version étendue de cette étude.

Pour consulter les processus d'entraînement (*training process*) et les matrices de confusion (*confusion matrix*) effectués dans ce projet, veuillez bien cliquer sur le lien suivant :

 [Training result.pdf](#)